



Huckel program

Goal:

The goal of the program is to provide a simple implementation of the Huckel approach for the description of polyenes. It allows to include dimerizations by alternating bonds or atom types (or both simultaneously). The program itself builds the Huckel matrix from scratch, following the users preferences. It asks for the length of the chain, the type (linear or cyclic), if dimerization is required and which type, letting the user to decide for the parameters. Given the distribution of eigenvalues' energy depends by the relative difference of bond length ($\Delta\beta$), a value of beta is kept fixed, while only the second one can be arbitrarily changed by the user. The values of alternating alpha can be both set by the user according the experimental parameters.

The program diagonalizes the Huckel matrix created by the user and derive the HOMO-LUMO gap according to the distribution of the eigenstates. It exploits a subroutine that selectively computes such a gap depending on the polyene's parameters such as its type (linear or cyclic) and the amount of electrons (i.e., the amount of carbons). The analyses of the HOMO-LUMO gap provides a first hint about the metallic/insulator behaviour of the molecule.

For linear polyenes the Total Position Spread tensor (TPS) is computed too. The value of the TPS assumes a scalar form for 1D polyenes, while its trends against the length of the chain expresses how such a property change as the dimension of the polyene increases. Both HOMO-LUMO gap and the TPS value (for linear molecules) are printed to the screen and explicitly stated each run. Furthermore, the program automatically handles the storage of such a data. For each run eigenvalues, eigenvectors, HOMO-LUMO gap values and TPS values are stored in personalized/folders (see here). The folder contains a second folder `Huckel_aut/` that contains an automatized version of the program used to generate plots and iteratively run the program (see here).

Description:

The description of the program will pass by each file that can be found in this folder. When the file is a Fortran file, the description of its content and explanation of its usage and goal. For building files (e.g., *makefile*), postprocessing of the results or data files a brief explanation is provided.

Main program (`Huckel_mat.f90`)

Variables declaration

```
! INITIALIZATION OF THE VARIABLES
```

This part of the program sets the local variables used in the main program, any local variable exclusively used in the subroutines is not displayed here.

Variables:

- `H(:, :)` : Huckel matrix.
- `eigen(:)` : Eigenvalues vector.
- `beta1, beta2` : β_1, β_2 in atomic units (Hartree), respectively.
- `lambda` : non-normalized TPS.
- `mu` : updates lambda over the atomic orbitals iterations.
- `n_coup` : number of electrons
- `at1, at2` : α_1, α_2 in atomic units (Hartree), respectively.
- `uw` : writing unit
- `d` : length of the chain
- `i, j, k` : counters.
- `t` : chain type (linear or cyclic)
- `filename` : stores the name of the saving files

Type and dimension settings (by the user)

```
! GRAPHICAL INTERFACE
```

This section is devoted to provide a first graphical interface of the program. This is the first interactive part of the program and it aims to store the parameters to build the Huckel matrix, here type and the length of the chain are collected, while dimerization parameters are gathered in the *filling routine*. It initializes it and defines the program stating its title as well as the author. Secondly, it asks to the user for the preferred type of the polyene. The program allows the user to enter only two input values: `'L'` for linear, `'C'` for

cyclic. The program handles entering an unexpected alphabetical input through a `do loop`, when the `( t .eq. 'C' .or. t .eq. 'L' )` is met, the loop ends and the program continues to the next interacting part. When an unexpected value for the polyene type is entered an error message is provided stating the provided input is unexpected and asks for a correct input.

The second interacting part asks the user for the dimension of the polyene. The program expects a even number, an approach identical to the one for the type with an appropriate condition is used to handle unexpected odd dimensions. As for the type, the program returns an explanation of the source of error and guides the user to provide a correct value. If an alphabetical (or non-numerical) the program stops and arises an error "*Bad integer for item 1 in list input*". `d` is declared as integers, assigning it a non-integer value generates an error.

The choice of restricting `d` to an even number comes from the observation that an even number of electrons represents a radical system, whose description is beyond the scope of this program. Indeed, as a radical, the physics of the system, its Huckel matrix, the arrangement of the eigenstate differ by a neutral molecule.

Memory allocation of variables

```
! MEMORY ALLOCATION
```

The Huckel matrix and the eigenvalues vector dimensions are assigned.

`Huckel(d,d)` is a $d \times d$ matrix, while `eigenv(d)` store d eigenvalues. The matrix is filled as the next step by the *filling subroutine*, while `eigenv` is filled as a consequence of the diagonalization in the *diagonalization subroutine*.

Calling the filling function & printing Huckel matrix

```
! MATRIX FILLING
```

In this branch of code the *filling subroutine* is called to build the Huckel matrix according to the parameters collected in the **type and dimension settings section**. Once the matrix is built and returned to the main code, a `do loop` is used to pass through its elements and print it line by line in a file called `Huckel_matrix.txt`. Since in fortran each time `write` is called the file line is

incremented by one, to maintain the matrix format an implicit do is used inside an outer explicit loop:

```
do i=1, d
  write(10,*) (H(i,j), j=1, d)
end do
```

Huckel matrix diagonalization: subroutine call

```
! HUCKEL MATRIX DIAGONALIZATION
```

This branch of code only calls the *matrix diagonalization* subroutine

HOMO-LUMO calculation: subroutine call

```
! HOMO-LUMO GAP CALCULATION
```

This branch of code only calls the subroutine implemented to compute the *HOMO-LUMO gap* case by case.

Storing eigenvalues and eigenvectors: subroutine call

```
! EIGENVALUES AND EIGENFUNCTIONS PRINTINGS
```

In this section the program calls a function devoted to save and store eigenvalues and eigenvectors in the appropriate folder, whose name is defined depending on the parameters collected and the form of the Huckel matrix.

TPS calculation

```
! TPS CALCULATION
```

Here, the program distinguishes between cyclic and linear systems. If a cyclic system run is being performed the program stops arising a "error 1" stop signal. Before stopping, the program prints the warning "The computation of TPS for cyclic polyenes is not required". TPS stands for total position spread, whose goal is to define how much delocalized is the GS. To do so, is required

to introduce a position operator, whose evaluation results difficult for cyclic structure, where the positions of the centers are arranged in a cyclic symmetry. Linear chains allows to set the origin of the system as position $x=1$. Assuming the orthogonalization of the atomic orbitals, the position matrix for a linear chain polyene assumes a diagonal form, where the i -th element is given by i .

When the system is linear, then, the calculation began stating that the calculation procedure has been initialized. Next, the amount of electron pairs is computed by halving the total number of electrons. Lambda, that will store the final non-normalized TPS, is initialized to 0 to avoid any strange value automatically assigned to the function.

The computation procedure is built by using three loops, according to the TPS formula:

$$\Lambda = \sum_{\substack{i \in \text{occ} \\ j \in \text{virt}}} \left(\sum_{\mu, \nu} c_{\mu i} c_{\nu j} \langle \chi_{\mu} | \hat{a} | \chi_{\nu} \rangle \right)$$

The outer loop runs over i , the intermediate one over j , while the third one over μ, ν . To note that, each i and j values are associated to a column in the eigenvectors matrix, i.e. the MOs, while μ, ν are the "rows" of such columns, corresponding to the AOs used to expand the MOs. Each term stores a coefficient, which is associated to the μ -th (ν -th) atomic orbital. Moreover, the fundamental assumption here is the orthogonalization of atomic orbitals. Even if explicitly distinguished, $\mu = \nu$, while $c_{\mu i} \neq c_{\nu j}$. Each pair (i, j) contributes to μ , whose role is to store the square root of the pairwise contribution, and to update Λ once the internal cycle is completed.

This procedure is repeated over the whole set of virtual orbitals. The algorithm proposed focuses on a single occupied orbital, and computes the contribution with respect to it, iterating over the full virtual set. When the set is terminated, the program moves to the next occupied orbital and repeats the full procedure on this too. When all the contributions for any virtual-occupied combinations have been computed, the entire set of μ values have been added to Λ .

Once the actual TPS value is obtained, it gets normalized diving by the size of the chain. Next, TPS values is printed with a message an explicite message. Not only the value of the TPS is shown directly during the run, but its value is also stored in an file, named after the properties of the system. The form of the

name of the file is standard: `TPS_lin_delta_alpha2`, thus it depends by the choices of the user. The name handling is accomplished using 'write' with a character variable (`filename`) devoted to store the value of the file. The path to find the information about the TPS of the run is also printed using the same file.

Finally `eigen` and `H` are deallocated.

Matrix filling subroutine (`subroutine.f90`)

The next subroutine is designed to fill the Huckel Matrix. The idea of designing a routine to perform this action follows from the willing to keep a clean main program. Overloading it with a huge amount of 'open', 'write', and 'read' may make it difficult to understanding, while breaking such a confusing part simplifies the readability of the program. Only two variables are declared as local, both `logical`: `bond_alt` and `atom_alt`. Alternation information are useful to fill the matrix, but their value is explicitly expressed through the matrix form and shape, their value is not required to further steps/analysis of the program or in the main program, thus they are only declared locally. To note that there is no gain by not calling the subroutine since it is required for the program to work, rather the partition was used, as mentioned, to keep a cleaner code. `t`, along with `at1`, `at2`, `beta1`, and `beta2` are required to be a global variable, because will be used later in other subroutines. The decision to declare `H`, `d`, `t`, `at1`, `at2`, `beta1`, and `beta2` as global variables derives from the fact that they are used in almost any step of the program.

Variables declaration

Input variables:

- `d`: length of the chain
- `t`: chain type (linear or cyclic)

Local variable:

- `bond_alt`: (logical). If `.True.`, the bond length is alternated ($\beta_1 \neq \beta_2$).
- `atom_alt`: (logical). If `.True.`, the atom type is alternated ($\alpha_1 \neq \alpha_2$).

- `i, j`: counters

Output variables:

- `H(:, :)`: Huckel matrix.
 - `beta1, beta2`: β_1, β_2 in atomic units (Hartree), respectively.
 - `at1, at2`: α_1, α_2 in atomic units (Hartree), respectively.
-

Bond length assignation

! ASSIGNATION OF THE BOND LENGHTS

Here, both values of β are automatically assigned. In the model we're using, $\beta_1 = -1.00$ is fixed, while the user can change the β_2 value. The initialization to the same value avoid to assign the same value of β_1 to β_2 later using a if. The `if` used later would have been anyway used, with or without this choice of pre-assigning the β_2 value.

Bond alternation choice & assignation

! BOND ALTERNATION CHOICE

Another graphical interface is created here, asking to the user if it is required to include bond alternation dimerization in the system. To avoid any wrong entry, here instructions are provided before to be read by the user. On the other hand, no error handling is included here.

The program initializes the bond alternation choice providing instruction on the allowed entries, and specify that any entry different from 'True' or 'False' will lead the program to crash. An if it is used to check whether the user entered `True` or `False`, in the former case the programs ask the user for the second bond length (i.e., β_2).

Atom alternation choice & assignation

! ATOM ALTERNATION CHOICE

This part of code defines whether the dimer contains the same atomic center. As in the previous step the programs ask whether the polyene has dimers basic units with alternating atomic centers. If so, the programs keep asking the atomic type. To note that this time, both α_1 and α_2 are assigned by the user. In facts, here, no assumptions are made about the default atomic type. When no alternation is observed, then `at1` and `at2` are set to be both equal to the value assigned by the user, otherwise they're set to different values. In both cases, the user decides the nature of the atomic center of the polyene.

The structure of this branch of code is similar to the previous. First thing first, the program asks whether the user requires bond alternation, if yes it proceeds

to ask both the on-site energies, otherwise asks for only one of them and assigns the same value to the second one. Indeed, the atom types are not assigned before, as occurred for β_1 and β_2 . The first value of alpha is always required, thus no `if` structures nor conditions controls that part of the program. If `atom_alt` is set to true by the user, the next if condition is satisfied and the program allows the user to manually assign α_2 , too. A further check is performed: if the user enters the same value for α_1 and α_2 , no error is raised, but a warning signaling that choice is displayed. Choosing to introduce atomic dimerization, and entering the same α value won't prevent the program to compile, nor to provide a reliable result, but there won't be any atomic dimerization when it comes to the calculations.

Filling procedure

! FILLING PROCEDURE

Here the main challenge is to properly fill the Huckel matrix without exceeding its dimension. To avoid any memory error (due to erroneous indexing) the filling procedure is split in 2 steps, the filling of the diagonal and then the assignment of off-diagonal terms

The reasoning for the diagonal filling is pretty simple: element on the diagonal alternates (if `atom_alt` is true), otherwise they have the same value. To use a single do to fulfill both the alternating and not alternating case, the do is designed assuming the atom types as if they were different. If atom types do not differ: `at1=at2` and the condition does not affect the filling cause, otherwise the same condition allows to alternate the center in the case of `atom_alt` being true. The choice to assign α_1 to odd index terms, and α_2 to even terms is arbitrary.

The reasoning in the off-diagonal filling part is based on the observation that any row refers to an atom. In the matrix a linking must then be repeated two times (C_1 is linked to C_2 , but also C_2 is linked to C_1). As a further example, if atom 2 and 3 are linked, the `H(2,3)` and the `H(3,2)` element will have the same value. The loop runs over $d - 1$ atoms. Indeed, the last line was already filled by the `H(d-1,d)`.

The third part of the program introduces, where required, open boundary conditions. The `(1,d)` and `(d,1)` elements are filled outside of the diagonal or off-diagonal loops. Those terms are coupled, as shown by the indexing, thus

carry the same values, whose assignation is accomplished if `t.eq. 'C'`. If the condition is not satisfied, then the matrix is linear and those elements are 0. Again, the filling procedure "does not fills" the $(i - 1)$ elements, but rather double fills `H(i,i+1)` and its simmetric elements `H(i+1,i)` (which is equivalent).

HOMO-LUMO gap routine

This subroutine is devoted to compute the HOMO-LUMO gap. Since the distribution of the eigenvalue varies according to the chain type (linear or cyclic) and the ring type (aromatic or antiaromatic), some case testing was introduced to chose which approach to use in order to compute the HOMO-LUMO gap. When dimerization is present, a band gap is generated that separates HOMO and LUMO, so the computation of the band is easily accomplished by the $E_{\text{HOMO}} - E_{\text{LUMO}}$. When it comes to cyclic non dimerized chains (i.e., rings), the lack of dimerization does not contribute to the generation of any band gap. In the aromatic case ($N_{\text{el}} = 4n + 2$) the distribution of the electrons in the eigenvalues generates only doubly-occupied orbitals, thus the last occupied pair of orbitals will contain the HOMO, while the first empty one is the LUMO. Their gap is computed as for the previous cases. When moving to the antiaromatic case ($N_{\text{el}} = 4n$), the last couple of electrons is arranged in two degenerate orbitals, HOMO and LUMO according to the index label, both HOMO when it comes to the actual energy, both singly occupied. It follows that the energy is computed by changing the index reference, not anymore by `eigen(n_coup+1)-eigen(n_coup)`, but with the equation `eigen(n_coup+2)-eigen(n_coup+1)`. Where in the majority of the case `eigen(n_coup)` represents the HOMO and `eigen(n_coup+1)` the LUMO, while in the latter case the LUMO, given the aforementioned distirbution, is `eigen(n_coup+2)`. More details about the eigenvalue spectrum are provided in the report.

Variable declaration

Input varibales:

- `d` : length of the chain
- `t` : chain type (linear of cyclic)
- `ratio` : absolute value of betas ratio $\left| \frac{\beta_1}{\beta_2} \right|$

Local variable:

- `n_coup` : number of electrons
- `filename` : stores the name of the file where the HOMO-LUMO gap is stored
- `typ1, typ2` : stores the casted value of `at1`, and `at2`.

- `at1, at2` : α_1 , α_2 in atomic units (Hartree), respectively.
- `eigen(:)` : Eigenvalues vector.

Variable initialization

```
! VARIABLE INITIALIZATION
```

Here, the first step is to define how many electron couples are in the system which is used to define the position of HOMO-LUMO. In linear, dimerized, and non-dimerized aromatic system the HOMO lies exactly in the half of `eigen`. Consider that the amount of eigenvalues corresponds to `d`, which is also the number of electrons. the amount of couples is `d/2`, then the HOMO falls exactly at half the size of `eigen`. On the other hand, for non-dimerized antiaromatic molecules the `n_coup` orbital is degenerate to the `n_coup+1`, thus a slightly different approach is used to compute the HOMO-LUMO gap. Here, the degenerate couple with index `n_coup, n_coup+1` is considered as both HOMO, each singly-occupied and the HOMO-LUMO gap was obtained computing the energy difference between `eigen(n_coup+2)-eigen(n_coup+1)`.

In order to simplify gathering the information the code handles itself displaying and saving the energy gap value in a properly named file. After initializing the amount of electronic couples, some casting is performed to convert the numerical atomic type (i.e., α_1 , α_2), originally a number that stands for the on-site energy, in a string (`character(len=8)`).

```
! EXPLICIT CASTING: FLOATING TO CHAR
write(typ1,'(F8.3)') at1
typ1=adjustl(typ1)
write(typ2,'(F8.3)') at2
typ2=adjustl(typ2)
```

The `write(typ*, 'format')` command tells the program to write inside `typ*` the content of `at1`, using a format indicated by `'format'`. The obtained variable `typ*` is a char used to define the name of the file where the HOMO-LUMO gap is stored

HOMO-LUMO calculation

! VARIABLES INITIALIZATION

Storing the HOMO-LUMO gap is performed in the same way regardless the case considered, but the name of the file differ for cyclic and linear case. Once the casting is performed the program meets a `select` statement whose case decision is driven by `t`. If the system is a linear chain, then the `case ('C')` is not satisfied, thus the `case (default)` section is entered. The `default` is not required when `select` is used, but it recommended to avoid unexpeced behaviour of the statement and it ensure at least one command is execeuted. When the chain is linear the program first print the gap to screen using the `eigen(n_coup+1)-eigen(n_coup)` approach, then builds the file name to store such a value in it. The name of the file is generated by usign an identical approach to the one exploited to cast.

```
write(filename, '(A,F4.2)') 'gap_lin_//trim(typ1)//"_"//trim(typ2)//"_"', ratio
```

The `write` statement contains the variable where to localize the path, the `'(A,F4.2)'` part specifies the kind and the format of the elements read and merge them in `filename`. Next a writing unit is open and associated with the file and the same value displayed before is now stored computing the same difference. The same approach is exploited for the cyclic case, but after the `select`, here a `if-else if-else` structure was built to deal with the different possibilities when it comes to ring. The condition handles the case of dimerized rings, regardless the source of dimerization is bond or atomic alternation. If it is so the program comutes the difference by the usual method `eigen(n_coup+1)-eigen(n_coup)`, then it generates the file name. If the chain is not dimerized, (i.e., no band gap), the program handles the different cases of antiaromatic and aromatic rings that requires a differe HOMO-LUMO gap calculation approach. Both cases computes the energy gap accordingly to the previous discussion due to the corresponding distribution of eigenvalues, then it generates the name of the file where the energy gap is stored, and it writes it in the file.

The `filename` structure for the linear case is composed by a string part that retains the reference to the chain type `"gap_lin"`, then the reference to the first atom type (`typ1`, i.e., α_1), and to the second one (`typ2`, i.e., α_2), separated by a `_`. The last element of the name is the ratio $|\frac{\beta_1}{\beta_2}|$, passed as argument to the subroutine. In the file are written both the gap and the corresponding

dimension of the chain. The unit open to write appends the gap each time the program is run, it follows that one can visualize how the gap evolves over elongating the size as the parameters are fixed, or also to confront different files with different parameters for the same `d` value. The file it ends up being: *"gap_lin_alpha1_alpha2_ratio"*

For the cyclic case, equivalent files are built for the same dimerization parameters, hence the type (aromatic or antiaromatic) are stored in the same value. Here, as for the linear case, `filename` carry an initial reference to the chain type *"gap_cyc"*, then, as discussed for the linear case the reference to α are displayed with the same format mentioned above, and lastly the ratio. The name of the file ends up being: *"gap_cyc_alpha1_alpha2_ratio"*. To note that, aromatic and antiaromatic systems are stored together in the same file. By contrast non-dimerized chains are stored in their own files whose reference is expressed or by α values, or `ratio`.

The subroutines concludes by closing the unit of the file. Since the different cases use the same unit, which is open only one time each subroutine called according to the case matched, closing the unit may be executed outside of the `select`. Finally, the last line of the program execute a command that displays on the screen the name of the file and tells the user where the gap information is stored, so that each run the user can directly find the generated information.

Saving eigenvectors and eigenvalues

Variable declaration

Input variables:

- `d` : length of the chain
- `t` : chain type (linear or cyclic)
- `ratio` : absolute value of betas ratio $\left| \frac{\beta_1}{\beta_2} \right|$
- `at1, at2` : α_1, α_2 in atomic units (Hartree), respectively.
- `eigen(:)` : Eigenvalues vector.
- `H(:, :)` : Huckel matrix.
- `filename` : stores the name of the eigenvector file
- `dirpath` : contains the full folder path where the eigenvectors are saved
- `which` : stores the reference to the chain type
- `fullpath` : used to store the eigenvalues file
- `typ1, typ2` : stores the casted value of `at1`, and `at2`.

Local variable:

- `uw` : writing unit updated cyclically

- `folder` : stores the casted value of the dimension
- `i,j` : counters

Variable initialization

! CASTING FROM DIGITS TO CHARACTER/STRING

The first part of the program is used to convert the original numerical value passed as input arguments to string because such values will be used as parameters to build the directory path where the eigenvectors obtained by the diagonalization, along with eigenvalues, are saved. As for the HOMO-LUMO gap subroutine, `write` statements are used to cast float into characters. Each write command writes in the first argument passed to it, a variable, a char obtained by converting the original variable that follows the command in a string/character version, the format is used to parse the variable to convert and to identify its kind. Each `write` statement is followed by a `adjustl`, whose role is to trim any initial space, it does the opposite of `trim`, that trims the trailing white spaces. `adjustl` is used to adjust the length of the converted character, avoiding blank spaces altering the path and affecting the saving procedure. `which` is devoted to store the type of the chain, thus its assignation of value depends on `t`, if it is "L", `which` is "linear", otherwise is "cyclic".

Creating the folder path

! CREATION OF THE FOLDER PATH

`dirpath` is devoted to store the directory path that will later add the name of the file `"eigenvalue.txt"` or `"eigenvector_ i "`. Each run generates a series of file including a `"eigenvalue.txt"` that contains the value of the eigenvalues of the system, each paired to its normalized index so that the file is ready to be plot as it is. Furthermore, the program generates, exploiting the current subroutine, the eigenvectors of the system originated by the diagonalization. Each one is stored in a different file, hence each `"eigenvector_ i"` contains the i